— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 5

8 Concurrent and Distributed Systems (mk428)

happens-before

(a) State the definition of the *happens-before* relation in a distributed system. [2 marks]

Answer: The happens-before relation captures causality in a distributed system. We say "a happens before b" or $a \to b$ iff any of the following is true:

- Event a and event b occurred in the same process, and a was executed before b.
- Event a is the sending of message m, and event b is the receipt of the same message m.
- There exists another event c such that $a \to c$ and $c \to b$.

Lamport clocks

(b) Describe how to compute Lamport timestamps. [2 marks]

Answer: Each process has a unique identifier p, and each process locally maintains a counter c. The Lamport timestamp of an event is the pair (c, p) at the time and process when that event occurred. Every time an event occurs, the counter is incremented. Every time a message is sent, the sender's current counter value is attached to the message. When a message is received, if the counter in the message is greater than the local counter of the recipient, the recipient's counter is increased to the value in the message, and then incremented.

Lamport clocks

(c) Lamport timestamps are often used to provide a total ordering on events in a system with multiple communicating processes. State how this total order is defined, and explain how this order relates to the happens-before relation. [2 marks]

Answer: For two Lamport timestamps (c_1, p_1) and (c_2, p_2) we define a total order $<$ as:

$$(c_1, p_1) < (c_2, p_2) \iff (c_1 < c_2) \vee (c_1 = c_2 \wedge p_1 < p_2).$$

While the happens-before relation is a strict partial order, this total order is a linear extension of happens-before. That is, for events a and b, if $a \to b$ and if $L(a)$ and $L(b)$ are the Lamport timestamps of those events, we have $L(a) < L(b)$. However, the converse is not true: when $L(a) < L(b)$ we do not know whether $a \to b$ or whether $a \parallel b$ (they are concurrent).

FIFO ordering; totally ordered multicast

(d) Explain what is meant by *FIFO broadcast* and *FIFO-total order broadcast* in the context of a process group. [2 marks]

Answer: FIFO broadcast: any message that is broadcast is delivered to each member of the group (provided it does not crash); and any messages sent by the same sender process are delivered in the order in which they were sent.

FIFO-total order broadcast: like FIFO broadcast, but additionally all processes deliver all messages in the same order.

process groups; receiving vs. delivering

(e) Give pseudocode for an algorithm that implements FIFO-total order broadcast using Lamport clocks. You may assume that each process has a unique identifier, and that the set of all process IDs in the group is known. Further assume

that the underlying network provides reliable FIFO broadcast; that is, you may use a function `sendFIFO(m)` that transmits message `m` to all processes in the group. The function `deliverFIFO(m)` is invoked at each process (including the sender) when message `m` is delivered at that process by the FIFO broadcast layer. Your algorithm should call `deliverTotalOrder(m)` with messages `m` in strictly monotonically increasing order of their Lamport timestamp.

Use this outline to get started:

```
// Called by the user when they want to send a message
function totalOrderBroadcast(msg) {
  let m = ...; // construct message to send by FIFO broadcast
  sendFIFO(m); // use underlying FIFO broadcast
}

// Called by the FIFO broadcast layer when a message is received
function deliverFIFO(m) {
  // Figure out the total order...
  // then call deliverTotalOrder(...) as appropriate
}
```

[10 marks]

Answer: Each process maintains its state in the following variables:

- `procs`: set of all process IDs in the group
- `proc`: ID of the current process
- `counter`: an integer, initially zero
- `minLamport`: a map where the keys are process IDs and values are integers (initial value 0 for each process)
- `holdback`: a priority queue where keys are (integer, process ID) pairs and values are messages; the `getMin()` method returns the entry with the smallest key according to the Lamport timestamp order defined in part (*c*).

```
// Called by the user when they want to send a message
function totalOrderBroadcast(msg) {
  counter++;
  let m = (counter, proc, msg);
  sendFIFO(m); // use underlying FIFO broadcast
}

// Called by the FIFO broadcast layer when a message is received
function deliverFIFO(m) {
  let (msgCounter, sender, msg) = m;
  if (counter < msgCounter) counter = msgCounter; // Lamport clock update
  counter++;
  holdback.add((msgCounter, sender), msg);
  minLamport[sender] = msgCounter; // latest timestamp seen from each process
  tryDelivery();
}
```

```
// Examines the holdback queue. Delivers any messages that are ready to the
// application, in increasing Lamport timestamp order.
function tryDelivery() {
  let threshold = getThreshold();
  while (!holdback.empty()) {
    let (timestamp, msg) = holdback.getMin();
    if (timestamp > threshold) break;
    deliverTotalOrder(msg);
    holdback.remove(timestamp);
  }
}

// Returns the threshold, which is the minimum latest timestamp across all
// processes. Due to FIFO ordering we know that all future messages delivered
// to deliverFIFO() will have a timestamp greater than this threshold.
function getThreshold() {
  let minimum = (+infinity, null);
  foreach p in procs {
    let timestamp = (minLamport[p], p);
    // The following comparison uses the total order on Lamport timestamps
    if (timestamp < minimum) minimum = timestamp;
  }
  return minimum;
}
```

failures

(*f*) Briefly comment on the fault-tolerance properties of your algorithm in Part (*e*). [2 marks]

Answer: This algorithm only delivers messages as long as all processes in `procs` are alive and sending messages. If a process stops sending further messages, e.g. due to a crash, then the threshold will no longer increase and so no more messages will be delivered.